

You are building AI Interior Atelier, a premium virtual interior design web application.

Use the attached PRD as the source of truth. Your job is not just to build screens. Your job is to build the product architecture in a way that supports a high-quality design intelligence layer over time.

The application should feel like a luxury virtual interior design studio, not a generic AI room generator.

Core product goal:

A user can create a home, add rooms, upload room photos, complete a design brief, receive a professional room diagnosis, generate three distinct mood board concepts, select one concept, create a shoppable product plan, generate realistic visual mockups, and refine the room through an in-app design chat.

Prioritize architecture, clean data modeling, modular AI service boundaries, and future extensibility.

Build the MVP in phases.

Phase 1: Foundation

Create the web app foundation using:

- Next.js
- TypeScript
- Tailwind CSS
- Supabase Auth
- Supabase Postgres
- Supabase Storage
- Vercel-ready deployment

Create a premium, calm, editorial UI that feels appropriate for a high-end design product.

Implement:

- Authentication
- Dashboard
- Home creation
- Home detail page
- Room creation
- Room detail page
- Photo upload
- Photo labeling
- Room tabs for Photos, Diagnosis, Mood Boards, Products, Renders, and Chat

Create database tables for:

- users
- homes
- rooms
- photos

- room_analyses
- mood_boards
- products
- renders
- revisions
- design_memories
- ai_runs or ai_debug_logs

Do not hard-code the product around one room. The first use case may be a home office, but the architecture should support many room types.

Phase 2: AI Service Boundaries

Before implementing real AI calls, create clean service files and placeholder outputs for:

- Room Vision Analyst
- Design Brief Interpreter
- Whole-Home Context Agent
- Style Director
- Mood Board Generator
- Product Sourcing Agent
- Scale and Fit Evaluator
- Render Prompt Director
- Design Critic
- Revision Agent
- Memory Agent

Each service should have:

- Clear TypeScript input types
- Clear TypeScript output types
- Zod validation schemas
- A placeholder mock implementation
- A future implementation boundary for OpenAI calls
- Error handling
- Logging to ai_runs or ai_debug_logs

The design intelligence layer should not be a single giant prompt. It should be a modular pipeline.

Phase 3: Design Intelligence Data Structures

Create strong internal schemas for:

Room analysis:

- room summary
- architecture
- doors
- windows
- flooring
- trim

- ceiling
- lighting
- existing items
- constraints
- opportunities
- uncertainties
- design risks
- recommended strategy

Mood boards:

- concept name
- design thesis
- style keywords
- palette
- materials
- furniture direction
- layout direction
- lighting direction
- art direction
- decor direction
- plant direction
- budget strategy
- why it works
- why user may reject it
- risk profile
- quality score

Products:

- category
- name
- retailer
- url
- image url
- price
- dimensions
- material
- finish
- style fit score
- scale fit score
- budget fit score
- material fit score
- luxury signal score
- reason selected
- risks
- alternatives

Renders:

- source photo
- selected mood board

- render prompt
- preservation constraints
- transformation instructions
- negative instructions
- critique
- quality score
- generated image URL

Revisions:

- user message
- classified revision type
- state before
- state after
- assistant response

Design memories:

- user-level preferences
- home-level design direction
- room-level selected decisions
- rejected styles
- approved products
- disliked products
- cross-room consistency notes

Phase 4: Style Library

Seed an internal design style library.

Include at minimum:

- Moody Coastal
- Organic Modern
- Transitional
- Modern Traditional
- European Country
- Lowcountry Coastal
- Masculine Executive
- California Casual
- Japandi
- Warm Minimal
- Classic Southern
- Collected Eclectic
- Boutique Hotel
- Modern Rustic

Each style should include:

- summary
- colors
- materials

- furniture silhouettes
- lighting types
- art direction
- plant direction
- luxury signals
- common mistakes
- budget substitutions
- styles it pairs well with
- styles it clashes with

The Style Director should use this style library to generate concepts, but the system should not merely copy a selected style. It should interpret the user's actual room, constraints, home context, and design brief.

Phase 5: Core UX Requirements

The Room Detail page should become the central workspace.

Tabs:

1. Photos
2. Diagnosis
3. Mood Boards
4. Products
5. Renders
6. Chat
7. Memory

The user should always understand:

- What room they are working on
- What stage the room is in
- What the selected concept is
- What has been saved
- What can be revised next

Add clear empty states for each stage.

Examples:

- "Upload room photos to begin your designer diagnosis."
- "Generate three design directions after the room has been analyzed."
- "Select a mood board before sourcing products."
- "Select a source photo before generating a mockup."

Phase 6: AI Pipeline Implementation

Once the app foundation and mock services are complete, replace the mock services with real AI calls one at a time.

Implement in this order:

1. Room analysis
2. Mood board generation
3. Mood board critique
4. Product plan generation
5. Scale and fit evaluation
6. Render prompt generation
7. Render critique
8. Room-aware design chat
9. Memory extraction

Each AI call should:

- Use structured outputs where possible
- Validate output against Zod schemas
- Save raw input and output
- Save prompt version
- Save model name
- Save quality score when applicable
- Fail gracefully
- Never corrupt room state on failure

Phase 7: Design Critic

Add a Design Critic function that scores outputs before the user sees them as final.

The critic should score:

- style clarity
- room fit
- functional fit
- scale realism
- color and material cohesion
- luxury signal
- originality
- practicality
- budget alignment
- whole-home alignment

Use this critic for:

- room diagnosis quality
- mood board quality
- product plan quality
- render quality

If an output is below threshold, regenerate once or show the issue in debug mode.

Phase 8: Product Sourcing

For MVP, start with a curated retailer system instead of trying to source from the entire internet.

Create a retailer config for:

- Wayfair
- West Elm
- CB2
- Crate & Barrel
- Pottery Barn
- Rejuvenation
- Lulu and Georgia
- McGee & Co.
- Article
- Target
- Amazon
- Etsy
- Chairish
- Serena & Lily
- Arhaus
- RH

For the first version, product sourcing can be mocked or semi-manual, but the product data model and UI should be ready for real sourcing.

Product recommendations must always include design rationale. The app should explain why each item works, not simply list products.

Phase 9: Mockup Studio

Create a Mockup Studio that lets the user:

- Select a source room photo
- Select the active mood board
- Generate a render prompt
- Generate or upload a mockup image
- View before and after
- Save the mockup
- Regenerate with instructions

For the first implementation, separate render prompt generation from actual image generation if needed. The app should still store the prompt, source image, selected concept, and intended transformation.

Phase 10: Room-Aware Design Chat

Create a persistent design chat for each room.

The chat should load:

- home context
- room brief
- room analysis
- selected mood board

- saved products
- saved renders
- design memories
- revision history

The chat should classify user messages into:

- general question
- style revision
- product revision
- render revision
- budget revision
- layout revision
- whole-home check
- memory update

The chat should be able to answer questions like:

- "Make this moodier."
- "Make this more coastal but not beachy."
- "Use my existing leather chair."
- "Find a cheaper rug."
- "Does this clash with the living room?"
- "What size art should go above this credenza?"
- "Should I use a bird of paradise or olive tree?"
- "Regenerate the render with darker walls."

Phase 11: Admin and Debug Tools

Create a hidden admin/debug mode.

It should show:

- AI runs
- prompt versions
- raw inputs
- raw outputs
- validation errors
- quality scores
- regeneration attempts
- token/cost estimates
- failed calls
- selected room state

This is critical because the quality of the product depends on prompt iteration and design output review.

Phase 12: Quality Standards

Do not ship generic design output.

All generated recommendations should feel:

- specific to the uploaded room
- aware of the user's taste
- aware of the room's function
- aware of architectural constraints
- practical to execute
- visually cohesive
- high-end
- well-rationalized

Avoid:

- generic "add a rug and plants" advice
- vague design language
- unrealistic furniture scale
- ignoring windows and doors
- ignoring budget
- overusing trend language
- copying a specific living designer
- producing mood boards that all feel the same
- recommending products without explaining why

Phase 13: Initial Build Instruction

Start by implementing Phase 1 only.

Do not implement real AI calls yet.

Build the full foundation, database schema, page structure, room workflow, photo upload system, and placeholder AI outputs. Keep the code modular so the AI services can be added incrementally.

After Phase 1 is complete, summarize:

- what was built
- what files were created
- what database tables were added
- what still needs environment variables
- what assumptions were made
- what the next recommended phase is

Build carefully, favor clean architecture over speed, and make the app easy to extend into a high-quality design intelligence platform.